

DFSha: Arquitectura distribuida y comunicaciones

Hito 2 | ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos, 2026-2
| Universidad EAFIT

Andrés Vélez Rendón

Jose Luis Restrepo

28 de septiembre de 2026

Índice general

1. Objetivo y alcance	1
Objetivo	1
Marco teórico	1
Alcance	2
2. Arquitectura implementada	2
Componentes	3
Organización del código	3
3. Algoritmos de particionamiento y distribución	3
Tamaño de bloque	3
Número de copias	4
Ubicación de los bloques	4
Capacidad	5
Integridad e idempotencia	5
Un solo escritor por archivo	5
Concurrencia y memoria	5
4. Comunicaciones	5
Protocolo por cada relación	5
Errores y reintentos	6
API del ControlNode, la consume el cliente	6
API del DataNode	7
API interna del clúster	8
Cómo funciona una subida	8
Cómo funciona una descarga	9

5. Entorno de ejecución	9
Docker Compose en una máquina	9
Nativo	10
Varias máquinas en AWS Academy	10
Parámetros	10
6. Pruebas y análisis de resultados	11
Pruebas automáticas	11
Demostración de punta a punta	11
Pruebas de escenarios	12
Medición de paralelismo	12
Análisis	13
7. Cambios respecto al Hito 1	13
8. Pendiente para el Hito 3	14
Notas	14
Referencias	15

Opción asignada: Opción 1, Cliente/Servidor, con el servicio distribuido entre varios nodos dentro de una red privada.

Este informe parte de la especificación del Hito 1 y describe lo que se construyó en el Hito 2. Las decisiones se enuncian en el cuerpo y se justifican en las **Notas** del final. Las notas del Hito 1 se citan como *Hito 1, N3*.

Código fuente: <https://github.com/AndresVelezR/DFSSha---Distributed-Systems>, etiqueta hito-2.

1. Objetivo y alcance

Objetivo

Convertir la especificación del Hito 1 en un sistema que corre. Un cliente, un ControlNode y varios DataNodes se comunican por red: los archivos se parten en bloques, los bloques se reparten y se replican entre DataNodes, y los datos nunca pasan por el ControlNode. Además, dejar fijado el contrato de comunicación de todas las relaciones entre componentes, incluidas las que se implementan en el Hito 3.

Marco teórico

Un sistema de archivos distribuido guarda archivos repartidos en varias máquinas y permite accederlos como si fueran un solo sistema. Hay dos enfoques. En el almacenamiento por objetos, como S3, la unidad es el archivo completo. En el almacenamiento por bloques, como GFS y HDFS, el archivo se parte y cada parte se

ubica por separado. DFSa es por bloques porque el enunciado pide que cada archivo quede repartido entre varios nodos y que haya concurrencia dentro de un mismo archivo (Hito 1, N1).

De GFS y HDFS se toman dos ideas. La primera es separar el plano de control del plano de datos. Un nodo maestro guarda solo metadatos y decide dónde va cada bloque, y los datos viajan directamente entre el cliente y los nodos de almacenamiento, así que el maestro no se vuelve cuello de botella con archivos grandes. La segunda es la escritura única: los bloques no se modifican nunca, así que dos copias de un bloque son idénticas por construcción y no hay que reconciliar réplicas (Hito 1, N8). Todo lo demás se construye desde cero; no se instala ni se envuelve Hadoop.

Alcance

El cronograma del curso asigna al Hito 2 el diseño e implementación de la arquitectura distribuida y la especificación de comunicaciones, y al Hito 3 la alta disponibilidad, la replicación, la consistencia y la seguridad. La tabla muestra qué cubre este hito de cada criterio de evaluación.

Criterio	En el Hito 2
Definición del servicio y diseño arquitectónico	Arquitectura maestro-trabajador ejecutable con un ControlNode y tres DataNodes, más un cuarto opcional (sección 2)
Especificaciones de comunicaciones	Contrato de las cinco relaciones entre componentes, con errores y reintentos. Cuatro están implementadas; la de ControlNode con ControlNode queda definida (sección 4)
Diseño para escalabilidad	DataNodes que se agregan con el sistema en marcha, tamaño de bloque que se adapta al número de nodos y ubicación por ocupación (secciones 3 y 6)
Replicación y consistencia	Réplica de bloques entre DataNodes, escritura única y un solo escritor por archivo. La réplica de metadatos y la reparación de copias son del Hito 3
Alta disponibilidad	Lectura desde otra réplica cuando un DataNode falla. Los tres ControlNodes y la reparación automática son del Hito 3
Seguridad	Token del cliente y clave compartida del clúster. HTTPS, cifrado y usuarios reales son del Hito 3

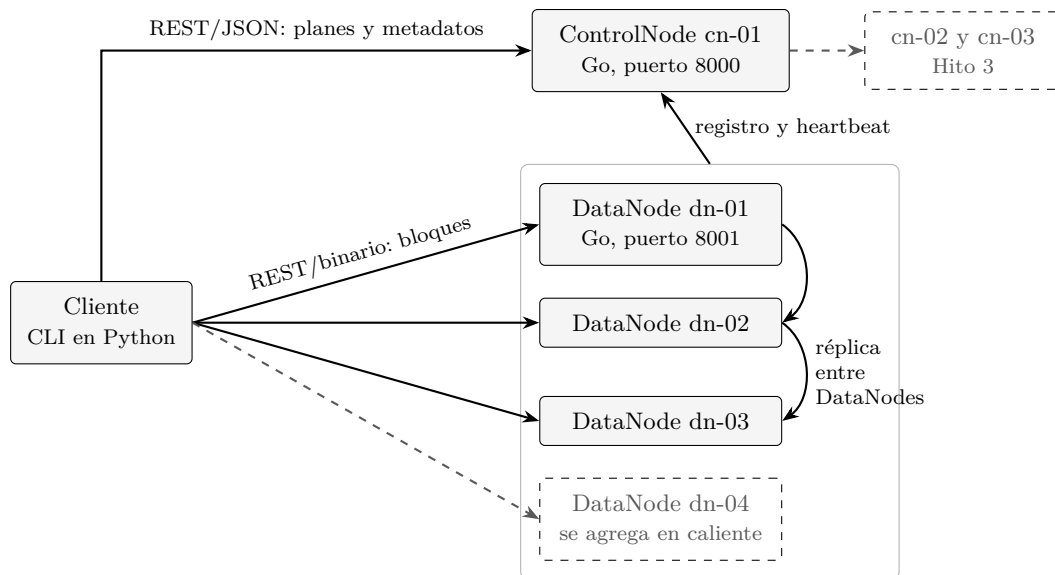
2. Arquitectura implementada

Tipo de arquitectura: Maestro y Trabajador, la definida en el Hito 1. El ControlNode coordina y sabe dónde está todo, y los DataNodes guardan los datos. El cliente le pide un plan al ControlNode y mueve los bloques directamente con los DataNodes.

En este hito hay un solo ControlNode activo. En el diagrama, los otros dos del diseño del Hito 1 aparecen en punteado: su réplica de metadatos y la elección de líder son del Hito 3, y sus endpoints ya existen y responden 501. El cuarto DataNode está apagado por defecto y sirve para demostrar el escalado.

Componentes

- **Cliente** (Python, sin dependencias externas). Programa de línea de comandos que se autentica, ejecuta los comandos del sistema de archivos, parte los archivos al subirlos y los reconstruye al bajarlos. No tiene direcciones de DataNodes en su configuración: las recibe en cada plan (Hito 1, N6).



- **ControlNode** (Go). Guarda en memoria el árbol de directorios, los metadatos de cada archivo y la lista de DataNodes con su ocupación. Calcula los planes de escritura y lectura, y publica un archivo solo cuando todos sus bloques se subieron.
- **DataNode** (Go). Guarda los bloques en su disco, verifica la firma de cada uno al recibirlo, los entrega completos o por rangos de bytes y los propaga a otros DataNodes. Se registra solo al arrancar y manda un heartbeat periódico con su ocupación.

Organización del código

Cada nodo es un binario independiente. Dentro de cada uno, el código se separa en archivos por responsabilidad, de modo que cada archivo tiene un solo motivo para cambiar [N1].

Archivo	Responsabilidad
controlnode/config.go	Parámetros operativos, leídos de variables de entorno y validados al arrancar
controlnode/planner.go	Decisiones de particionamiento y ubicación: tamaño de bloque, número de copias, capacidad y destinos. No depende de HTTP
controlnode/handlers_*.go	Endpoints agrupados por tema: autenticación, sistema de archivos, subida, descarga y clúster
controlnode/server.go	Estado en memoria y tabla de rutas
datanode/handlers.go	Endpoints de bloques
datanode/control_client.go	Comunicación con el ControlNode: registro y heartbeat
datanode/peer_client.go	Comunicación con otros DataNodes: propagación de copias
datanode/storage.go	Disco local: capacidad, bloques guardados y bytes que ocupan
client/dfsha.py	Cliente de línea de comandos

3. Algoritmos de particionamiento y distribución

Las decisiones de esta sección viven en `planner.go` como funciones puras y tienen pruebas automáticas (sección 6).

Tamaño de bloque

Se implementa la fórmula del Hito 1 (N3), con los parámetros leídos de la configuración:

```
B = clamp( block_size_min , S / (k · N) , block_size_max )
```

donde **S** es el tamaño del archivo, **N** el número de DataNodes vivos y **k** los bloques mínimos por nodo, redondeando a la potencia de dos inferior. Con los valores por defecto (8 MiB, 64 MiB y **k** = 4):

Archivo	Bloque con 3 nodos	Bloques	Bloque con 4 nodos	Bloques
1 MB	8 MiB	1	8 MiB	1
40 MB	8 MiB	5	8 MiB	5
200 MB	8 MiB	24	8 MiB	24
500 MB	32 MiB	15	16 MiB	30
1 GiB	64 MiB	16	64 MiB	16
10 GiB	64 MiB	160	64 MiB	160

La fila de 500 MB muestra la adaptación al clúster: con un nodo más, el bloque se reduce a la mitad para que cada nodo siga recibiendo al menos **k** bloques.

Número de copias

Se aplican las dos reglas del Hito 1 (N4 y “Qué pasa al quitar un nodo”), en este orden:

1. El factor configurado nunca supera **N - 1**, donde **N** es el número de DataNodes registrados en el clúster, para que siempre exista un nodo donde rehacer una copia perdida [**N2**].
2. Si hay menos DataNodes vivos que ese factor, se guardan las copias posibles en vez de rechazar la escritura, y el ControlNode lo avisa en su registro.

Registrados	Vivos	Factor configurado	Copias por bloque
3	3	2	2
3	2	2	2
3	1	2	1, con aviso
2	2	2	1
3	3	3	2
10	10	3	3

Las copias de un mismo bloque siempre quedan en DataNodes distintos.

Ubicación de los bloques

Se usa la asignación voraz al nodo menos cargado, el *list scheduling* de Graham (Hito 1, N5). Para cada copia de cada bloque se elige el DataNode vivo con menor porcentaje de ocupación que no tenga ya una copia de ese bloque. La implementación agrega tres detalles:

- **Carga virtual.** Mientras se arma el plan, a cada nodo elegido se le suma el tamaño del bloque, para que los bloques de una misma subida no caigan todos en los nodos que estaban más vacíos al empezar [N3].
- **Desempate por identificador** de nodo, para que el mismo estado produzca siempre el mismo plan [N3].
- **Ocupación medida por bloques.** Cada DataNode reporta los bytes que ocupan sus bloques sobre la capacidad de su volumen, y no el espacio usado del disco [N4].

Capacidad

Antes de entregar un plan, el ControlNode comprueba que el archivo, multiplicado por su número de copias, quepa en el espacio libre sumado de los DataNodes vivos. Si no cabe, responde **507** y no reserva la ruta, en vez de dejar que la subida falle a mitad de camino (Hito 1, “Sobre el tamaño máximo de archivo”).

Integridad e idempotencia

Cada bloque se identifica por la firma SHA-256 de su contenido. El DataNode recalcula la firma al recibirlo y lo rechaza si no coincide. Lo escribe primero a un archivo temporal y lo renombra al final, así que nunca queda visible un bloque a medias. Al descargar, el cliente vuelve a verificar la firma de cada bloque.

Subir un bloque es idempotente: si el DataNode ya lo tiene, no lo reescribe, pero sí lo propaga a los destinos indicados [N5].

Un solo escritor por archivo

Pedir un plan de subida reserva la ruta durante `write_lease_ttl`, que es de 60 s por defecto. Mientras la reserva esté vigente, otro plan sobre la misma ruta recibe **409**. Si el cliente muere, la reserva vence sola. El archivo solo se publica cuando el cliente confirma todos sus bloques, así que un archivo a medio subir nunca es visible (Hito 1, N8).

Concurrencia y memoria

El cliente sube y baja hasta cuatro bloques en paralelo (opción `--workers`, Hito 1 N10), y cada DataNode atiende cada conexión en una goroutine. Cada worker lee del disco solo su bloque, y al bajar escribe cada bloque directamente en su posición del archivo de salida. Así, en memoria nunca hay más de *workers* × *tamaño de bloque* bytes, sin importar el tamaño del archivo [N6].

4. Comunicaciones

Todo es **REST sobre HTTP** (Hito 1, N11): el control viaja en JSON y los bloques como cuerpo binario sin codificar. El cliente envía su token en la cabecera **Authorization: Bearer** a todos los nodos, y los nodos entre sí se identifican con la clave del clúster en **X-Cluster-Key**. En este hito se usa HTTP; el contrato no cambia al activar HTTPS en el Hito 3. El ControlNode escucha en el puerto **8000** y cada DataNode en el **8001**.

Protocolo por cada relación

Relación	Expone	Protocolo	Para qué	Estado
Cliente y ControlNode	ControlNode	REST/JSON	Autenticación, sistema de archivos, planes de escritura y lectura, confirmación	Implementada
Cliente y DataNode	DataNode	REST/binario	Subir y bajar bloques	Implementada
DataNode y DataNode	DataNode	REST/binario	Propagar la copia al escribir	Implementada
ControlNode y DataNode	ControlNode	REST/JSON	Registro, heartbeat y reporte de bloques	Implementada; las órdenes de reparación que viajan en la respuesta del heartbeat son del Hito 3
ControlNode y ControlNode	ControlNode	REST/JSON	Réplica de metadatos y elección de líder	Contrato definido; responde 501 hasta el Hito 3

Errores y reintentos

Situación	Comportamiento
Fallo de red al subir un bloque	Hasta 3 reintentos contra el mismo DataNode, esperando 1, 2 y 4 s. Es seguro porque la operación es idempotente [N7]
Fallo al bajar un bloque, o firma incorrecta	Se pasa de inmediato a la siguiente réplica, sin reintentar la misma
Respuesta de error del servidor	No se reintenta: no es un fallo de red sino una decisión del servidor
La reserva venció antes de confirmar (410)	El cliente pide un plan nuevo y repite la subida una vez; los bloques ya subidos no se duplican
Un bloque sin copias vivas al pedir el plan de lectura	El ControlNode responde 503

API del ControlNode, la consume el cliente

Operación	Solicitud	Respuesta	Errores
POST /auth/login	{username, password}	200 {token, expires_in}	401
GET /fs/ls?path=	sin cuerpo	200 {entries:[{name, type, size, blocks, replicas_ok}]}	404
POST /fs/mkdir	{path}	201	409 ya existe, 404 sin padre
DELETE /fs/rmdir?path=	sin cuerpo	204	409 no vacío, 404
DELETE /fs/rm?path=	sin cuerpo	204	404
POST /fs/mv	{src, dst}	200	404, 409 destino existe
GET /fs/stat?path=	sin cuerpo	200 {size, block_size, n_blocks, replicas_ok, owner}	404
POST /files/upload/plan	{path, size}	200 con el plan de escritura	409 reserva activa, 507 sin espacio, 404 sin padre, 503 sin DataNodes
POST /files/upload/commit	{upload_id, blocks:[{index, block_id, size, stored_on}]}	200 {status:"publicado"}	409 faltan bloques, 410 reserva vencida
POST /files/upload/abort	{upload_id}	204	ninguno
GET /files/download/plan?path=	sin cuerpo	200 con el plan de lectura y las réplicas vivas	404, 503 bloque sin copias

replicas_ok es el número de copias en DataNodes vivos del bloque menos replicado del archivo, así que baja apenas un DataNode deja de mandar heartbeat. Este es el plan de escritura del archivo de la demostración, con los destinos que asignó el ControlNode; el identificador de subida y la fecha son ilustrativos:

```
POST /files/upload/plan -> { "path": "/demo/original.bin", "size": 40000000 }
200 OK
{ "upload_id": "3f9c2a7e5b1d4c08", "block_size": 8388608, "n_blocks": 5,
  "lease_expires_at": "2026-09-28T05:20:12Z",
  "blocks": [
    { "index": 0, "targets": [ {"node":"dn-01","host":"dn1","port":8001},
                              {"node":"dn-02","host":"dn2","port":8001} ] },
    { "index": 1, "targets": [ {"node":"dn-03","host":"dn3","port":8001},
                              {"node":"dn-01","host":"dn1","port":8001} ] },
    ... ] }
```

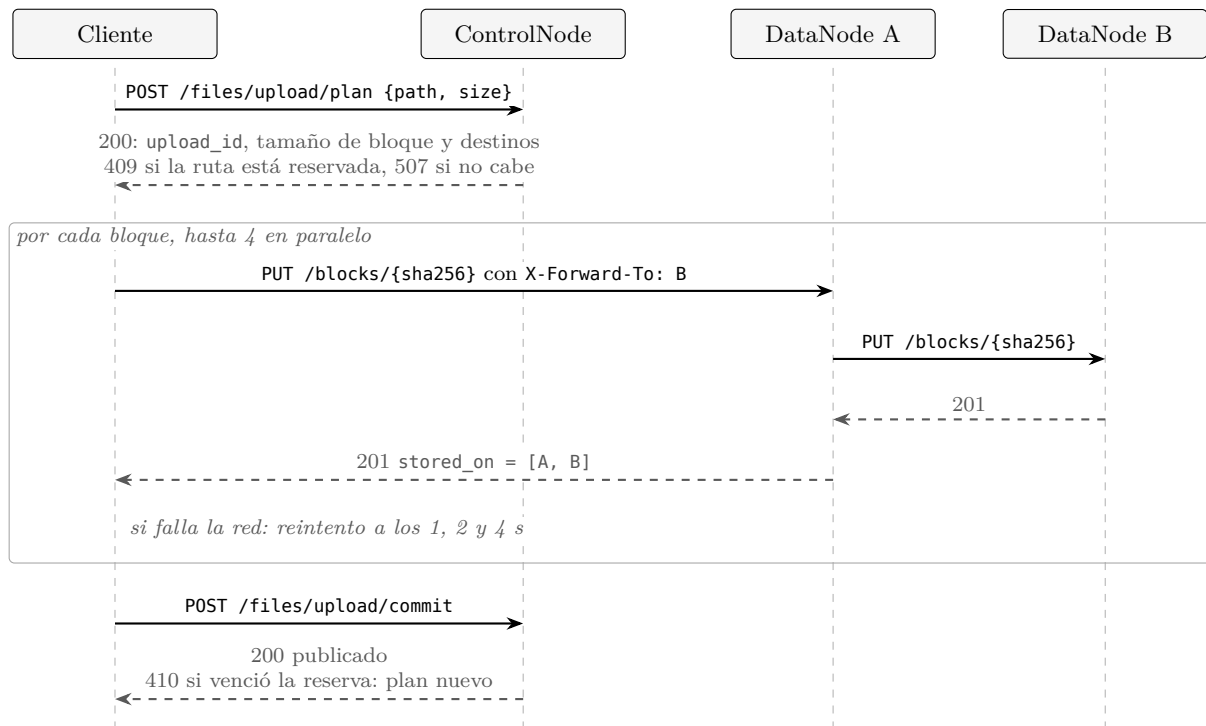
API del DataNode

Operación	La consume	Solicitud	Respuesta	Errores
PUT /blocks/{id}	Cliente, DataNode	Cuerpo binario. Si hay que propagar, X-Forward-To con las URL de los otros destinos y X-Forward-Node-i con sus identificadores	201 si lo creó, 200 si ya existía; {block_id, size, stored_on}	401, 409 firma incorrecta
GET /blocks/{id}	Cliente, DataNode	Opcionalmente Range	200 binario, 206 parcial	401, 404
DELETE /blocks/{id}	ControlNode	X-Cluster-Key	204	401, 404
POST /blocks/{id}/replicate	ControlNode	{target:{host, port}}	202	401, 502
GET /health	Operación	sin cuerpo	200 {node_id, capacity, used, n_blocks}	ninguno

API interna del clúster

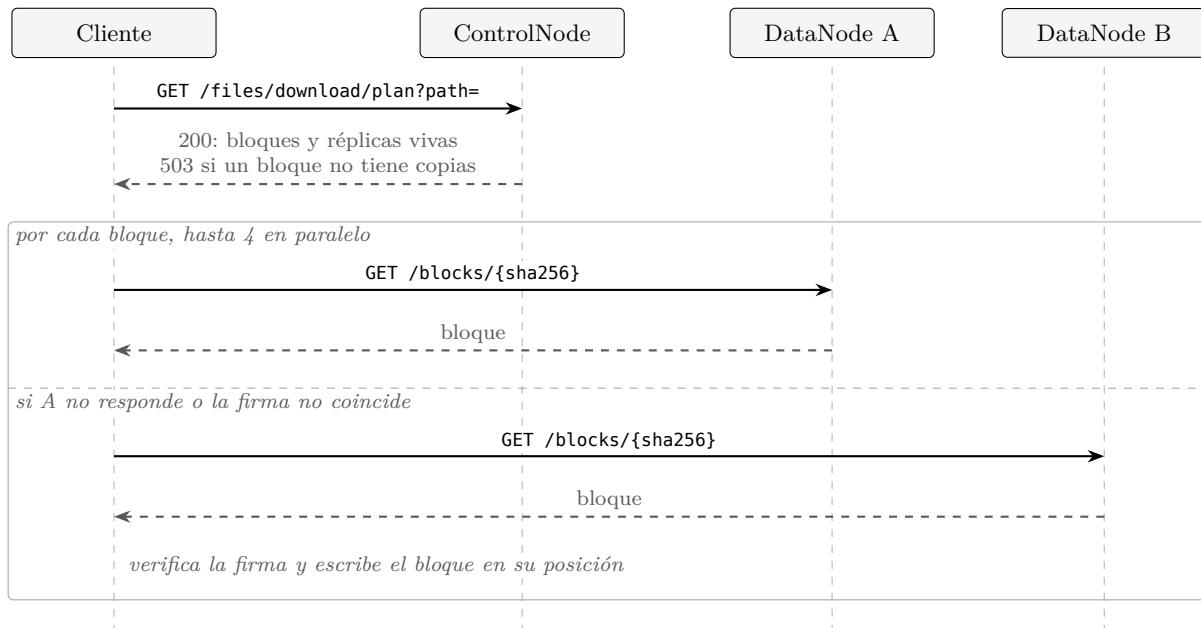
Operación	Quién la usa	Para qué
POST /cluster/register	DataNode → ControlNode	Alta al arrancar con {node_id, host, port, capacity, used}. Se repite sola si el ControlNode deja de reconocer al nodo
POST /cluster/heartbeat	DataNode → ControlNode	Señal de vida cada 3 s con {node_id, capacity, used}. La respuesta trae {orders:[]}, vacía hasta el Hito 3
POST /cluster/blockreport	DataNode → ControlNode	Inventario {node_id, blocks:[id]}. Contrato fijado; el DataNode lo empieza a enviar en el Hito 3
POST /cluster/replicate	Entre ControlNodes	Réplica de metadatos. Responde 501 hasta el Hito 3
POST /cluster/election	Entre ControlNodes	Elección de líder con {node_id, log_length}. Responde 501 hasta el Hito 3

Cómo funciona una subida



1. El cliente pide el plan indicando ruta y tamaño.
2. El ControlNode valida la ruta, comprueba que no esté reservada y que el archivo quepa, calcula tamaño de bloque, número de copias y destinos, y reserva la ruta. El archivo queda en curso, no publicado.
3. El cliente lee cada bloque del disco, calcula su firma y lo envía al primer destino, hasta cuatro en paralelo.
4. El DataNode verifica la firma, guarda el bloque y lo propaga en cadena al siguiente destino, para que el cliente suba cada bloque una sola vez (Hito 1, N12).
5. El cliente confirma la subida con la lista de bloques y dónde quedó cada copia.
6. El ControlNode publica el archivo. Si la subida se interrumpe, el cliente la aborta o la reserva vence, y el archivo nunca se publica.

Cómo funciona una descarga



1. El cliente pide el plan de lectura.
2. El ControlNode devuelve, para cada bloque, las réplicas que están en DataNodes vivos.
3. El cliente descarga los bloques en paralelo. Si una réplica no responde o entrega una firma incorrecta, pasa a la siguiente.
4. Cada bloque verificado se escribe en su posición de un archivo temporal, que solo toma el nombre final cuando llegaron todos.

5. Entorno de ejecución

Cada nodo corre en un contenedor Docker o de forma nativa, y expone su propia API REST. Hay tres formas de levantar el sistema, y las tres se probaron (sección 6).

Docker Compose en una máquina

Servicio	Qué es	Puerto
controlnode	ControlNode cn-01	8000, publicado
dn1, dn2, dn3	DataNodes dn-01 a dn-03, cada uno con su volumen	8001, en la red interna
dn4	Cuarto DataNode, apagado por defecto (perfil scale)	8001, en la red interna
client	Cliente, como herramienta (perfil tools)	ninguno

```
docker compose up -d --build controlnode dn1 dn2 dn3
./scripts/demo.sh # demostración completa del hito
```

Nativo

```
go build -o controlnode ./cmd/controlnode && ./controlnode
go build -o datanode ./cmd/datanode
NODE_ID=dn-01 PORT=8001 STORAGE_DIR=./dn1 ./datanode # uno por DataNode
python3 client/dfsha.py login demo demo
```

Varias máquinas en AWS Academy

La infraestructura del Hito 1 (sección 5) queda preparada en la carpeta **deploy/aws** del repositorio: un solo archivo de Docker Compose para todas las VMs y un archivo **.env** por VM, que dice qué corre en ella y con qué dirección se anuncia a los demás.

VM	Contenedores en el Hito 2
vm-01	ControlNode cn-01 y DataNode dn-01
vm-02	DataNode dn-02 ; cn-02 se agrega en el Hito 3
vm-03	DataNode dn-03 ; cn-03 se agrega en el Hito 3
vm-04	DataNode dn-04 , apagada hasta la demostración de escalado

El puerto 8001 debe quedar abierto también hacia el cliente, porque el cliente sube y baja los bloques directamente de los DataNodes [N8].

Parámetros

Todos los parámetros que usa la lógica implementada se leen de variables de entorno al arrancar, así que se cambian sin recompilar. Un valor inválido detiene el arranque con un mensaje que dice qué variable está mal.

Parámetro del Hito 1	Variable de entorno	Defecto	Estado
replication_factor	REPLICATION_FACTOR	2	Implementado
block_size_min	BLOCK_SIZE_MIN_MB	8	Implementado
block_size_max	BLOCK_SIZE_MAX_MB	64	Implementado
blocks_per_node	BLOCKS_PER_NODE	4	Implementado
heartbeat_interval	HEARTBEAT_INTERVAL_SECONDS	3	Implementado
heartbeat_misses	HEARTBEAT_MISSES	3	Implementado: un nodo se da por muerto a los 9 s
write_lease_ttl	WRITE_LEASE_TTL_SECONDS	60	Implementado
client_parallel_blocks	opción --workers del cliente	4	Implementado
availability_target	ninguna		Ver el cambio 3 de la sección 7
node_full_threshold	ninguna		Hito 3, con el rebalanceo
rebalance_threshold	ninguna		Hito 3, con el rebalanceo

Parámetro del Hito 1	Variable de entorno	Defecto	Estado
<code>blockreport_interval</code>	ninguna		Hito 3, con la reparación de copias
<code>cn_heartbeat_interval</code>	ninguna		Hito 3, con los tres ControlNodes

No se exponen variables para funciones que todavía no existen, porque serían configuraciones sin efecto.

6. Pruebas y análisis de resultados

Pruebas automáticas

Se ejecutan con `go test ./...` y `python3 -m unittest discover client`. Solo usan la biblioteca estándar de cada lenguaje.

Qué se prueba	Casos
Planificador	Tamaño de bloque (mínimo, máximo y redondeo), número de bloques, número de copias en los seis escenarios de la tabla de la sección 3, capacidad del clúster, copias en nodos distintos, reparto parejo y preferencia por el nodo más vacío
Endpoints de subida	Segunda reserva sobre la misma ruta: 409 . Reserva vencida: se libera. 3 DataNodes: 2 copias; 2 DataNodes: 1 copia. Archivo que no cabe: 507 sin dejar la ruta reservada
Sistema de archivos	replicas_ok baja cuando un DataNode deja de mandar heartbeat
DataNode	La ocupación cuenta solo bloques completos, no escrituras en curso
Cliente	Reintentos a los 1 y 2 s hasta tener éxito; 4 intentos y luego falla; un error del servidor no se reintenta; un 410 pide un plan nuevo una sola vez; bloques escritos en desorden reconstruyen el archivo exacto

Demostración de punta a punta

`scripts/demo.sh` levanta el clúster con Docker Compose y ejecuta la secuencia completa. Resultado de la última ejecución:

Paso	Resultado
Levantar el clúster	El ControlNode reporta 3 DataNodes vivos
Subir un archivo de 40 MB	5 bloques de 8 MiB, cada uno con 2 copias en DataNodes distintos. <code>ls</code> y <code>stat</code> muestran replicas_ok: 2
Descargarlo	Archivo reconstruido con la misma firma SHA-256 que el original
Dos planes seguidos sobre la misma ruta	200 y luego 409
Apagar <code>dn1</code> y descargar	A los 9 s el ControlNode reporta 2 DataNodes vivos. Los bloques que estaban en <code>dn-01</code> se leen de su otra réplica y la firma sigue siendo idéntica

Paso	Resultado
Agregar dn4 en caliente	El ControlNode pasa a 4 DataNodes vivos sin reiniciar nada. En la siguiente subida, dn-04 , que llega vacío, recibe una copia de cada uno de los 5 bloques

Pruebas de escenarios

Escenario	Resultado
Subida con un DataNode destino caído que vuelve a los 3 s	El cliente registra “reintento 1/3 en 1s” y “reintento 2/3 en 2s”, y el bloque se sube en el tercer intento
3 DataNodes registrados y 1 caído	El plan asigna 2 copias por bloque entre los 2 vivos
Clúster de solo 2 DataNodes	El plan asigna 1 copia por bloque
BLOCK_SIZE_MIN_MB=4 al levantar	El bloque pasa a 4 MiB sin recompilar
REPLICATION_FACTOR=dos	El ControlNode no arranca y reporta que el valor no es un entero
Tres VMs simuladas	Tres proyectos de Docker Compose sin red en común, con deploy/aws y cada DataNode en un puerto propio. Se comunican solo por la IP de la máquina, como entre VMs. El cliente nativo sube y baja un archivo de 30 MB y la firma coincide
Ejecución nativa	ControlNode y 3 DataNodes como procesos, sin Docker; subida y descarga correctas

Medición de paralelismo

Sobre las tres VMs simuladas se subió y bajó un archivo de 200 MB (24 bloques de 8 MiB, 2 copias) con 1 y con 4 workers, tres veces cada uno. La tabla muestra la mediana.

Workers	Subida	Caudal de subida	Bajada	Caudal de bajada
1	0.71 s	280 MB/s	0.57 s	348 MB/s
4	0.36 s	558 MB/s	0.39 s	513 MB/s

Con 4 workers la subida es 1.97 veces más rápida y la bajada 1.46 veces. La mejora confirma que el paralelismo por bloques funciona: mientras un bloque viaja, otro se lee y se firma. Los números absolutos no representan una red real, porque las tres VMs comparten una máquina y el tráfico no sale de ella. La medición sobre AWS queda para el Hito 4, como previó el Hito 1 (N10) [N9].

Análisis

- **Separación de planos.** Los registros del ControlNode muestran solo peticiones de control. Los bloques viajan entre el cliente y los DataNodes, y entre DataNodes, así que el tamaño de los archivos no pasa por el ControlNode.
- **Reparto.** En la subida de 40 MB, las 10 copias quedaron repartidas 4, 3 y 3 entre los tres DataNodes, y ningún bloque tuvo sus dos copias en el mismo nodo.

- **Nodo nuevo.** Al agregar `dn-04`, las subidas nuevas lo prefieren hasta emparejar su ocupación. Es el autobalanceo por escrituras que describe el Hito 1: un nodo nuevo empieza a recibir bloques de inmediato, sin mover los que ya estaban guardados.
 - **Hallazgos corregidos durante las pruebas.** La primera ejecución del escalado no envió bloques a `dn-04` por dos causas. En Docker local los DataNodes medían su ocupación sobre el disco compartido de la máquina [N4]. Además, `dn4` usaba una imagen de Docker propia que no se reconstruía con las demás, así que corría código viejo. Ahora la ocupación se mide por bloques y todos los DataNodes comparten una imagen. También se corrigió que ejecutar el cliente con Docker Compose volvía a encender un DataNode apagado a propósito, lo que invalidaba la prueba de fallos [N10].
 - **Copias degradadas.** Si la propagación de una copia falla durante la subida, por ejemplo porque el segundo destino se cae en ese momento, el bloque se publica con las copias que se lograron, como indica el Hito 1. `ls` y `stat` lo muestran en `replicas_ok`, y reponer la copia faltante es trabajo de la reparación del Hito 3.
-

7. Cambios respecto al Hito 1

El Hito 1 previó que cualquier ajuste se documentaría en los hitos siguientes (RNF8). Estos son los cambios de decisión frente a lo planteado. Lo que simplemente se implementa en un hito posterior está en la sección 8.

1. `cd` no se implementa como comando con estado de sesión. El Hito 1 lo definió como un comando que maneja el cliente. El cliente construido no guarda estado: cada comando es un proceso nuevo y no hay una sesión que recuerde un directorio actual. La navegación se resuelve pasando la ruta completa a cada comando (`ls`, `stat`, `put`, `get`, `mv`). Cumple el mismo propósito sin agregar un estado local que se pueda desincronizar del servicio.

2. Las primitivas de RF3 no se exponen como comandos ni endpoints separados. `open`, `read` por rango, `write`, `lock` y `close` siguen siendo el modelo que fundamenta `put` y `get`. El plan de subida es el `open` con `lock`, porque reserva la ruta. Cada `PUT` de bloque es un `write`, y el commit es el `close`. El DataNode ya soporta lectura parcial con la cabecera `Range`. Exponerlas como superficie aparte no lo exige ningún criterio de este hito y agregaría una API que habría que mantener y asegurar en el Hito 3. Se reconsiderará si un hito posterior exige acceso aleatorio explícito a los archivos.

3. Los parámetros se leen de variables de entorno, y `availability_target` no se expone. El Hito 1 pedía que ningún valor estuviera en el código y que se cambiaran sin recompilar, y eso se cumple. Las variables de entorno son el mecanismo nativo de Docker Compose, y un archivo `.env` junto al `docker-compose.yml` cumple el papel del archivo de configuración sin un formato adicional que leer y validar. El factor de replicación se configura directamente con el valor que da la fórmula de disponibilidad (Hito 1, N4). Recalcularlo en cada arranque no aporta nada, porque el resultado solo cambia cuando cambia el objetivo, que es justo cuando se ajusta la variable.

4. Subir un bloque que ya existe responde 200, no 409. El Hito 1 especificaba “409 ya existe y se trata como éxito”. En HTTP, un `PUT` es idempotente y repetirlo con el mismo contenido es una operación exitosa, así que 200 describe mejor lo que pasa y evita que el cliente tenga que interpretar un código de error como éxito. Un bloque nuevo sigue respondiendo 201.

5. El puerto 8001 también se abre hacia el cliente. La tabla de infraestructura del Hito 1 abría hacia el cliente solo el puerto 8000. Como el cliente sube y baja los bloques directamente de los DataNodes, también necesita el 8001 [N8].

8. Pendiente para el Hito 3

Queda para el hito de Alta Disponibilidad, Replicación, Consistencia y Seguridad:

- **Tres ControlNodes** con réplica de metadatos y elección de líder. `/cluster/replicate` y `/cluster/election` responden 501.
- **Persistencia de metadatos** en disco. Hoy viven en memoria y se pierden si el ControlNode se reinicia.
- **Reparación de copias.** El campo `orders` de la respuesta del heartbeat existe pero siempre va vacío, y el DataNode todavía no envía su inventario de bloques. Si un DataNode muere, o si falla la propagación de un bloque durante una subida, ese bloque se queda con menos copias hasta que exista la reparación.
- **Verificación de copias en el commit.** Hoy el ControlNode confía en la lista `stored_on` que reporta el DataNode. Contrastarla con el inventario de bloques es parte de la reparación.
- **Rebalanceo** de bloques ya guardados al agregar un nodo, y `node_full_threshold`.
- **Recolección de bloques huérfanos** tras `rm` o una subida abortada.
- **Seguridad:** HTTPS, cifrado de bloques en disco y control de acceso real por usuario. Hoy hay un único usuario de demostración.
- **Despliegue en AWS Academy** sobre las VMs del Hito 1, con la configuración ya preparada en `deploy/aws`.

Notas

[N1] **Por qué archivos por responsabilidad y no capas.** Cada servicio tiene unos cientos de líneas. Una arquitectura hexagonal, con puertos y adaptadores, agregaría interfaces con una sola implementación cada una, porque hoy no hay un segundo almacenamiento de metadatos ni un segundo transporte que intercambiar. Lo que sí hacía falta era poder probar las decisiones del sistema sin levantar un servidor, y eso se logra dejando el planificador como funciones puras en su propio archivo. El punto en que una interfaz sí se justifica es el Hito 3: cuando los metadatos pasen de memoria a un registro persistente y replicado, el acceso a ellos tendrá dos implementaciones reales.

[N2] **Por qué el tope $N - 1$ cuenta nodos registrados y no vivos.** El Hito 1 impone $N - 1$ para que siempre quede un nodo donde rehacer una copia perdida, y eso es una propiedad del tamaño del clúster. Si se aplicara sobre los nodos vivos, con 3 DataNodes y uno caído el sistema guardaría una sola copia teniendo dos nodos disponibles. Bajaría la redundancia justo cuando ya ocurrió una falla, y contradiría la regla del mismo Hito 1 de que, con nodos caídos, se guardan las copias que se puedan.

[N3] **Por qué carga virtual y desempate.** La ocupación de cada nodo llega con el heartbeat cada 3 segundos, así que todos los bloques de un plan se deciden sobre la misma foto. Sin carga virtual, los 16 bloques de un archivo de 1 GiB irían a los mismos dos nodos, los más vacíos al empezar. Sumando el tamaño de cada bloque asignado, la heurística reparte dentro del mismo plan. El desempate por identificador existe porque Go recorre los mapas en orden aleatorio: sin él, dos nodos con la misma ocupación producirían planes distintos cada vez, lo que dificulta probar y explicar el sistema.

[N4] **Por qué la ocupación se mide por bloques.** El Hito 1 ubica cada bloque en el nodo con menor porcentaje de disco ocupado, y supone un volumen dedicado a bloques en cada VM. En ese volumen, el espacio usado del disco y los bytes de bloques coinciden. En Docker sobre una sola máquina, en cambio, todos los DataNodes ven el mismo disco, y el espacio usado refleja todo lo que hay en la máquina. En la prueba de escalado eso hizo que un nodo vacío pareciera tan lleno como los demás. Contar los bytes de los bloques da la medida correcta en ambos casos.

[N5] **Por qué se propaga un bloque que ya existe.** Antes, un DataNode que ya tenía el bloque respondía sin propagarlo. Un reintento después de una falla parcial, o un bloque con el mismo contenido en otro archivo, que por eso tiene el mismo identificador, quedaba registrado con una sola copia. Además, el DataNode ahora consume el cuerpo de la petición aunque no lo guarde, para que el cliente termine de enviarlo sin un error de conexión.

[N6] **Por qué memoria acotada.** La primera versión del cliente leía el archivo completo antes de subirlo y juntaba todos los bloques en memoria antes de escribirlos al bajar. Con un archivo de 10 GB en una instancia `t3.small`, que tiene 2 GB de memoria, eso es imposible, y contradice la idea central del Hito 1: un archivo grande nunca se carga completo en memoria. La descarga escribe sobre un archivo `.part` que se renombra al final, para que una descarga fallida nunca deje un archivo que parezca completo.

[N7] **Por qué se reintenta la escritura y no la lectura.** Así lo define el Hito 1, y tiene una razón. Al escribir, el plan tiene un único punto de entrada para cada bloque, el primer destino, así que la única forma de insistir es reintentar ese mismo destino. Al leer hay varias réplicas, y pasar a la siguiente es más rápido que esperar 1, 2 y 4 segundos a la misma. Solo se reintentan los fallos de red, en los que no hubo respuesta. Una respuesta de error del servidor es una decisión, no un fallo pasajero, y repetirla daría el mismo resultado.

[N8] **Por qué el puerto 8001 se abre hacia el cliente.** La separación de planos, que es el centro de la arquitectura, exige que el cliente hable directamente con los DataNodes. Con el 8001 cerrado hacia el cliente, solo podrían subirse archivos desde dentro de la red de AWS. Por la misma razón, cada DataNode se anuncia con la dirección que el cliente puede alcanzar: la IP privada si el cliente corre dentro de la red de AWS, y la pública si corre desde fuera.

[N9] **Por qué la medición es solo indicativa.** En una sola máquina, las transferencias no cruzan una red real y compiten por el mismo procesador y el mismo disco. Sirve para confirmar que el paralelismo reduce el tiempo, pero no para dimensionar el sistema. Ajustar `client_parallel_blocks` con datos reales requiere las VMs de AWS.

[N10] **Por qué el cliente de Docker no depende de los nodos.** Docker Compose enciende las dependencias de un servicio cada vez que se ejecuta. Con el cliente declarado como dependiente de los DataNodes, ejecutarlo después de apagar uno lo volvía a encender, y la prueba de tolerancia a fallos no probaba nada. El cliente es una herramienta que se usa con el clúster ya levantado, así que no declara dependencias.

Referencias

- S. Ghemawat, H. Gobioff y S.-T. Leung. *The Google File System*. ACM Symposium on Operating Systems Principles, 2003.
- K. Shvachko, H. Kuang, S. Radia y R. Chansler. *The Hadoop Distributed File System*. IEEE Symposium on Mass Storage Systems and Technologies, 2010.
- R. L. Graham. *Bounds for certain multiprocessing anomalies*. Bell System Technical Journal, 45(9), 1966.
- A. Vélez Rendón y J. L. Restrepo. *DFS_ha: Especificación del proyecto*, Hito 1, 2026. En `docs/hito-1`.